

CS402/MA492 Cryptography

Tobias Rossmann

School of Mathematics, Statistics and Applied Mathematics

NUI Galway

Lecture 7:

Cryptography in Python, part 2

Last time

- Jupyter notebooks
- Python basics:
 - arithmetic
 - conditionals
 - lists

Today

- More Python basics:
 - strings
 - loops
 - functions
 - files
- The Python module `cs402`

Strings

- String constants are indicated using single or double quotes.
- Python strings cannot be modified.
- Characters are retrieved using `[index]`.
- Concatenation (`+`) and `len` are supported by strings.
- Strings support numerous further methods.

In [2]:

```
s = 'this is ' + 'a string'
s
```

Out[2]:

```
'this is a string'
```

In [3]:

```
s[4]
```

Out[3]:

```
','
```

In [4]:

```
s.capitalize()
```

Out[4]:

```
'This is a string'
```

In [5]:

```
s.upper()
```

Out[5]:

```
'THIS IS A STRING'
```

In [6]:

```
s.replace(" is ", " isn't " )
```

Out[6]:

```
"this isn't a string"
```

Loops

We can loop over "iterables" (e.g. lists, strings) using **for loops**.

In [7]:

```
for c in 'a string':  
    print()
```

```
a
```

```
s  
t  
r  
i  
n  
g
```

In [11]:

```
for i in range(26):  
    print(i, i*i)
```

```
0 0  
1 1  
2 4  
3 9  
4 16  
5 25  
6 36  
7 49  
8 64  
9 81  
10 100  
11 121  
12 144  
13 169  
14 196  
15 225  
16 256  
17 289  
18 324  
19 361  
20 400  
21 441  
22 484  
23 529  
24 576  
25 625
```

A **while loop** repeatedly executes the code in its body until a condition is satisfied.

In [12]:

```
s = 'blah'  
while len(s) < 20:  
    s = s + s  
print(s)  
print(len(s))
```

```
blahblahblahblahblahblahblah  
32
```

- `break` can be used to immediately terminate execution of a loop.
- `continue` jumps right to the next iteration.

Functions

Functions are used to package isolated pieces of computational functionality for later use. They are defined using `def` statements.

In [13]:

```
def gcd(a, b):  
    if not b: # equivalent to `if b == 0:`  
        return a  
    return gcd(b, a % b)
```

In [14]:

```
gcd(12431324, 98468456)
```

Out[14]:

4

Files

Working with files in a program usually involves three steps:

1. Open the file.
2. Read from or write to the file.
3. Close the file.

Files are opened using `open` . The second parameter indicates whether the file is opened for reading (`'r'`) or writing (`'w'`).

Warning: a file opened for writing is truncated!

In [15]:

```
f = open('cs402.py', 'r')  
data = f.read() # Read entire file as a string  
f.close()  
len(data)
```

Out[15]:

9027

In [17]:

```
f = open('testfile.txt', 'w')  
f.write('test string')  
f.close()
```

[testfile.txt \(testfile.txt\)](#)

The module `cs402`

- The module `cs402` provides functionality related to the content of this course. It will be updated throughout the semester.
- To use it, you will need to download the file [cs402.py](http://www.maths.nuigalway.ie/~rossmann/cs402/cs402.py) (<http://www.maths.nuigalway.ie/~rossmann/cs402/cs402.py>) and put it somewhere where your Python installation can find it.
- `cs402` requires the Python libraries [NumPy](http://www.numpy.org/) (<http://www.numpy.org/>) and [SymPy](https://www.sympy.org/) (<https://www.sympy.org/>). These are included with [Anaconda](https://www.anaconda.com/) (<https://www.anaconda.com/>).

To load `cs402` , run

```
from cs402 import *
```

at the top of your code.

Alphabets

`cs402` defines a number of useful alphabets.

In [18]:

```
from cs402 import *
```

In [19]:

```
ALPHABET26
```

Out[19]:

```
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

In [20]:

```
ALPHABET27
```

Out[20]:

```
'ABCDEFGHIJKLMNOPQRSTUVWXYZ '
```

In [21]:

```
ALPHABET68
```

Out[21]:

```
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789.,;:\n'
```

Note. `'\n'` indicates the **newline character**.

Internally, `cs402` operates on (sequences of) numbers rather than characters. The correspondence is defined in terms of a given alphabet and made explicit using the functions `string_to_int_list` and `int_list_to_string`.

In [23]:

```
string_to_int_list(ALPHABET27, 'HELLO')
```

Out[23]:

```
[7, 4, 11, 11, 14]
```

In [24]:

```
string_to_int_list(ALPHABET68, 'HELLO')
```

Out[24]:

```
[33, 30, 37, 37, 40]
```

In [25]:

```
int_list_to_string(ALPHABET27, [3,2,1,0])
```

Out[25]:

```
'DCBA'
```

Modular arithmetic

In [26]:

```
gcd(44, 1199)
```

Out[26]:

```
11
```

In [27]:

```
gcd(27, 1199)
```

Out[27]:

```
1
```

In [28]:

```
a = 27
n = 1199
b = modular_inverse(a, n)
b
```

Out[28]:

```
977
```

In [29]:

```
a*b
```

Out[29]:

26379

In [30]:

```
(a * b) % n
```

Out[30]:

1

Ciphers

The class `Cipher` acts as a base class for various ciphers that we will use.

Shift ciphers

Let's create a shift cipher over our 27-character alphabet:

In [31]:

```
S = ShiftCipher(ALPHABET27)
```

The methods `encrypt_string` and `decrypt_string` can be used to do exactly what their names indicate.

The first argument should be the key (i.e. shift value for shift ciphers) and the second argument should be the plaintext or ciphertext, respectively.

In [32]:

```
plain = 'THIS IS A VERY SECRET MESSAGE AND SHOULD NOT BE READ BY ANYONE'
```

In [33]:

```
cipher = S.encrypt_string(12, plain)
cipher
```

Out[33]:

```
'ETUDLUDMLGQCJLDQOCQELYQDDMSQLMZPLDT FXPLZ ELNQLCQMPLNJLMZJ ZQ'
```

The function `frequency_histogram` displays a frequency histogram for a given string.

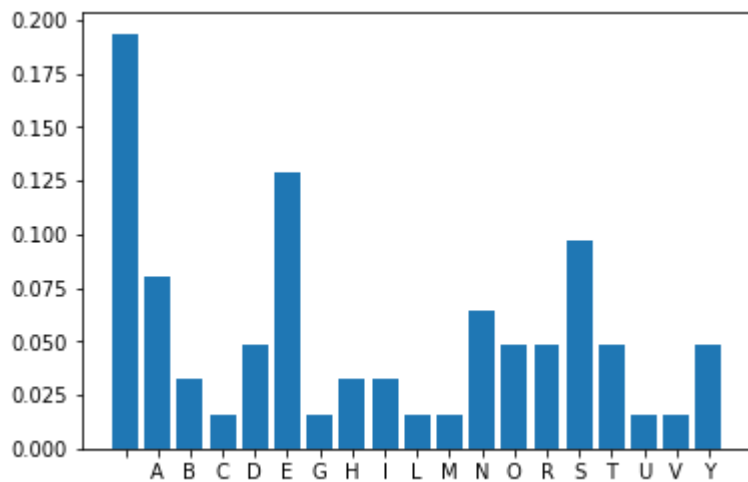
If you're using Jupyter notebooks, include the following somewhere:

In [34]:

```
%matplotlib inline
```

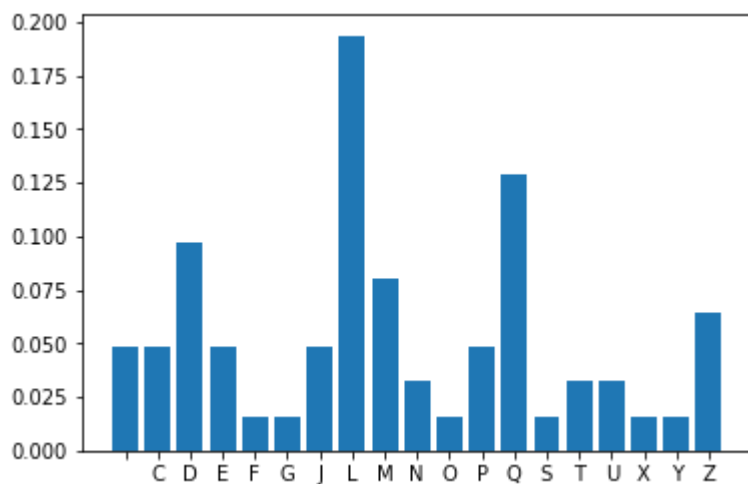
In [35]:

```
frequency_histogram(plain)
```



In [36]:

```
frequency_histogram(cipher)
```



- We know that space is the most frequently used character in English.
- Had we just received the ciphertext, we would have suspected that L corresponds to space.

In [37]:

```
ALPHABET27.index('L')
```

Out[37]:

11

In [38]:

```
ALPHABET27.index(' ')
```

Out[38]:

26

Since $(26 + 12) \% 27 == 11$, this suggests that the key used was 12.

Affine ciphers

In [39]:

```
A = AffineCipher(ALPHABET27)
```

Keys are pairs (a, b) , where a is required to have an inverse modulo the length of the alphabet.

In [40]:

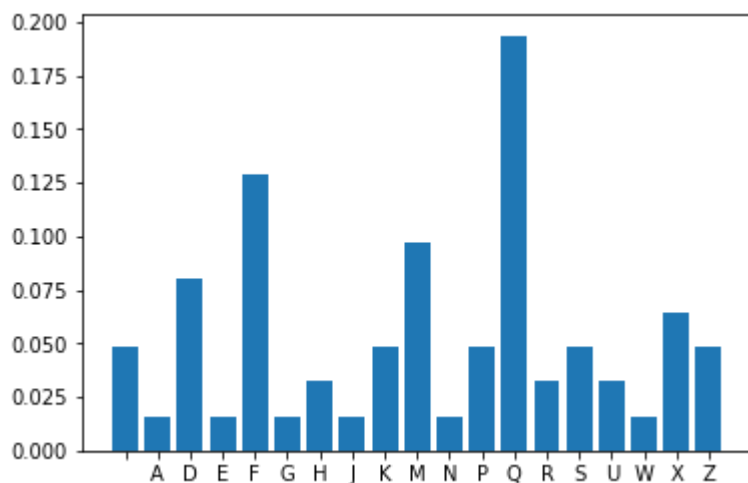
```
key = (14, 3)
cipher = A.encrypt_string(key, plain)
cipher
```

Out[40]:

```
' UHMQHMQDQAFZPQMFEZF QJFMMDGFQDXSQMUKNWSQXK QRFQZFDSQRPQDXPKXF '
```

In [41]:

```
frequency_histogram(cipher)
```



After space, E is the most common letter in English. If we only knew the ciphertext, we would suspect that Q corresponds to space and F corresponds to E. Let us try to recover the key from that.

Our guess (in terms of characters):

Plaintext	Ciphertext
space	Q
E	F

In [42]:

```
for c in [' ', 'Q', 'E', 'F']:
    print(ALPHABET27.index(c))
```

```
26
16
4
5
```

Our guess (in terms of integers):

Plaintext	Ciphertext
26	16
4	5

We thus need to find (a, b) with

$$\begin{aligned}26a + b &\equiv 16 \pmod{27} \\4a + b &\equiv 5 \pmod{27}\end{aligned}$$

Taking the difference, we obtain:

$$22a \equiv 11 \pmod{27}$$

and thus

$$a \equiv 11x \pmod{27},$$

where x is an inverse of 22 modulo 27.

In [43]:

```
a = (modular_inverse(22,27) * 11) % 27
a
```

Out[43]:

14

Once we know, a , we can recover b e.g. via $b \equiv 5 - 4a \pmod{27}$:

In [44]:

```
b = (5 - 4*a) % 27
b
```

Out[44]:

3

In [45]:

```
A.decrypt_string((14,3),cipher)
```

Out[45]:

'THIS IS A VERY SECRET MESSAGE AND SHOULD NOT BE READ BY ANYONE'

Vigenère ciphers

The following example is taken from Lecture 4:

In [46]:

```
V = VigenereCipher(ALPHABET27)
V.encrypt_string('BAD', 'GREETINGS')
```

Out[46]:

```
'HRHFTLOGV'
```

File access

All cipher object in `cs402` support the methods `encrypt_file` and `decrypt_file`.

Arguments: key, input file name, output file name.

Demonstration: [plain.txt](http://www.maths.nuigalway.ie/~rossmann/cs402/plain.txt) (<http://www.maths.nuigalway.ie/~rossmann/cs402/plain.txt>).

In [48]:

```
V = VigenereCipher(ALPHABET68)
```

In [49]:

```
V.encrypt_file('BAD', 'plain.txt', 'cipher.txt')
```

Ciphertext: [cipher.txt](http://www.maths.nuigalway.ie/~rossmann/cs402/cipher.txt) (<http://www.maths.nuigalway.ie/~rossmann/cs402/cipher.txt>).

In [50]:

```
V.decrypt_file('BAD', 'cipher.txt', 'decrypted.txt')
```

Decrypted: [decrypted.txt](http://www.maths.nuigalway.ie/~rossmann/cs402/decrypted.txt) (<http://www.maths.nuigalway.ie/~rossmann/cs402/decrypted.txt>).